

```
# Cell 1: Install Hugging Face Hub
!pip install -q huggingface_hub

print("✅ Hugging Face Hub installed!")
```

✅ Hugging Face Hub installed!

```
# Cell 2: Login to Hugging Face
from huggingface_hub import notebook_login

notebook_login()
```

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF\_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (<https://huggingface.co/settings/tokens>)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn()

```
# Cell 3: Create repository programmatically
from huggingface_hub import HfApi

api = HfApi()

repo_id = "ranasinghehashini/srilankan-food-recognition"

try:
    api.create_repo(
        repo_id=repo_id,
        repo_type="model",
        private=False # Public repository
    )
    print(f"✅ Repository created: https://huggingface.co/{repo_id}")
except Exception as e:
    print(f"Note: {e}")
    print("Repository may already exist - that's okay!")
```

✅ Repository created: <https://huggingface.co/ranasinghehashini/srilankan-food-recognition>

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
# Cell 4: List your model files
import os

SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition'

files_to_upload = {
    'best_model.pth': 'PyTorch model (for Python users)',
    'model_torchscript.pt': 'TorchScript model (for mobile apps)',
    'model_info.json': 'Model metadata',
    'confusion_matrix.png': 'Performance visualization',
    'per_class_accuracy.png': 'Per-class results',
    'training_curves.png': 'Training progress',
}

print("📁 Files ready to upload:")
print("="*70)
for filename, description in files_to_upload.items():
    filepath = os.path.join(SAVE_DIR, filename)
    if os.path.exists(filepath):
        size = os.path.getsize(filepath) / (1024 * 1024) # MB
        print(f"✅ {filename:30s} ({size:.2f} MB) - {description}")
    else:
        print(f"❌ {filename:30s} - NOT FOUND")
```

📁 Files ready to upload:

```
=====
✅ best_model.pth           (5.62 MB) - PyTorch model (for Python users)
❌ model_torchscript.pt     - NOT FOUND
❌ model_info.json          - NOT FOUND
✅ confusion_matrix.png     (0.30 MB) - Performance visualization
✅ per_class_accuracy.png   (0.25 MB) - Per-class results
✅ training_curves.png      (0.37 MB) - Training progress
```

```
# Cell 5: Generate Model Card
model_card = """---
license: mit
tags:
- food-recognition
- few-shot-learning
- prototypical-networks
- sri-lankan-cuisine
- computer-vision
- deep-learning
datasets:
- custom
language:
- en
metrics:
- accuracy
library_name: pytorch
---

# 🍌 Sri Lankan Food Recognition Model

## Model Description

A custom Prototypical Network trained to recognize vegetables across different cooking transformations in Sri Lankan cuisine.

### Key Features
- 📈 90.25% validation accuracy with limited training data
- 📖 Few-shot learning approach (requires minimal examples)
- 📱 Mobile-ready (TorchScript format included)
- 🍲 Specialized for Sri Lankan cooking transformations

## Performance Metrics

| Metric | Score |
|-----|-----|
| Validation Accuracy | 90.25% |
| Few-Shot Test Accuracy | 87.75% |
| Full Test Set Accuracy | 84.91% |
| Mean Per-Class Accuracy | 84.15% |
| Average Confidence | 85.32% |

## Model Architecture

- Type: Custom Prototypical Network
- Embedding Dimension: 128
- Input Size: 224x224 RGB images
- Training Method: Episodic few-shot learning (4-way 2-shot)
- Parameters: ~500K (lightweight!)

### Architecture Details
```
Conv Block 1: Conv2D(3→32) + BatchNorm + ReLU + MaxPool
Conv Block 2: Conv2D(32→64) + BatchNorm + ReLU + MaxPool
Conv Block 3: Conv2D(64→128) + BatchNorm + ReLU + MaxPool
Conv Block 4: Conv2D(128→256) + BatchNorm + ReLU + MaxPool
Global Average Pooling
FC: 256→256 + ReLU + Dropout(0.3)
FC: 256→128 + L2 Normalization
```

## Classes

The model recognizes 8 classes across different cooking states:

1. `carrot_raw` - Raw carrot (100% accuracy)
2. `carrot_white_curry` - Carrot in white curry (33.33% accuracy)
3. `greenbeans_raw` - Raw green beans (100% accuracy)
4. `greenbeans_tempered` - Tempered green beans (66.67% accuracy)
5. `greenbeans_white_curry` - Green beans in white curry (100% accuracy)
6. `pumpkin_raw` - Raw pumpkin (100% accuracy)
7. `pumpkin_red_curry` - Pumpkin in red curry (87.50% accuracy)
8. `pumpkin_white_curry` - Pumpkin in white curry (85.71% accuracy)

## Dataset

- Total Images: 331
- Training: 230 images (70%)
- Validation: 48 images (15%)
- Test: 53 images (15%)
- Images per Class: ~30-40

### Data Characteristics
```

```

- Captured under varied lighting conditions
- Multiple angles and presentations
- Authentic Sri Lankan cooking methods
- Controlled and real-world images

## Usage

### Quick Start (Python)
```python
import torch
from torchvision import transforms
from PIL import Image
from huggingface_hub import hf_hub_download

Define model architecture (same as training)
class PrototypicalNetwork(torch.nn.Module):
 # ... [architecture code]
 pass

Load model
checkpoint = torch.load(model_path, map_location='cpu')
model = PrototypicalNetwork(embedding_dim=128)
model.load_state_dict(checkpoint['model_state_dict'])
model.eval()

Preprocessing
transform = transforms.Compose([
 transforms.Resize((224, 224)),
 transforms.ToTensor(),
 transforms.Normalize([0.485, 0.456, 0.406], [0.229, 0.224, 0.225])
])

Inference
image = Image.open('your_food_image.jpg').convert('RGB')
image_tensor = transform(image).unsqueeze(0)

with torch.no_grad():
 embedding = model(image_tensor)
 # Compare with class prototypes for classification
 ...

Mobile Deployment (Flutter/React Native)

Use `model_torchscript.pt` with PyTorch Mobile:
```dart
// Flutter example
import 'package:pytorch_mobile/pytorch_mobile.dart';

final model = await ModelObjectDetection.loadModel('model_torchscript.pt');
final result = await model.getImagePrediction(imageFile, 224, 224);
```

Training Details

- Framework: PyTorch
- Training Duration: 52 epochs (~13 hours on CPU)
- Optimizer: Adam (lr=0.0001)
- Learning Rate Schedule: Step decay ($\gamma=0.5$ every 50 epochs)
- Loss Function: Prototypical loss (cross-entropy on negative distances)
- Data Augmentation: Horizontal flip, rotation ($\pm 15^\circ$), color jitter

Limitations

⚠️ Known Issues:
- `carrot_white_curry` has lower accuracy (33.33%) – likely due to visual similarity with other white curry items
- Small test set per class (6–8 images) may cause variability
- Trained specifically on Sri Lankan cooking methods – may not generalize to other cuisines
- Best performance with well-lit, clear images

Use Cases

✅ Ideal for:
- Dietary monitoring apps for Sri Lankan users
- Food recognition in mixed dishes
- Allergen detection in cooked foods
- Educational apps about Sri Lankan cuisine
- Restaurant menu assistance

Citation

If you use this model in your research or application, please cite:

```

```

```\bibtex
@misc{srilankan-food-recognition-2026,
  author = {Your Name},
  title = {Sri Lankan Food Recognition: A Transformation-Aware Deep Learning Approach},
  year = {2026},
  publisher = {Hugging Face},
  note = {Custom prototypical network for recognizing cooked vegetable ingredients}
}
```

Model Card Authors

- **Your Name** – Research & Development
- **Institution:** NIBM / Coventry University

License

This model is released under the MIT License. See LICENSE file for details.

Acknowledgments

- Research conducted as part of BSc (Hons) Computing final year project
- Dataset collected and annotated specifically for this research
- Inspired by prototypical networks (Snell et al., 2017)

Last Updated: February 2026

Save to file
with open('/content/README.md', 'w') as f:
 f.write(model_card)

print("✅ Model card created: README.md")
print("\n📄 Preview (first 500 characters):")
print(model_card[:500] + "...")

```

✅ Model card created: README.md

📄 Preview (first 500 characters):

```

license: mit
tags:
- food-recognition
- few-shot-learning
- prototypical-networks
- sri-lankan-cuisine
- computer-vision
- deep-learning
datasets:
- custom
language:
- en
metrics:
- accuracy
library_name: pytorch

```

# 🍌 Sri Lankan Food Recognition Model

**## Model Description**

A custom **\*\*Prototypical Network\*\*** trained to recognize vegetables across different cooking transformations in Sri

```

Cell 6: Upload everything to Hugging Face
from huggingface_hub import HfApi, upload_file, upload_folder
import os

Configuration
REPO_ID = "ranasinghehashini/srilankan-food-recognition"
SAVE_DIR = '/content/drive/MyDrive/Research-2026/SriLankanFoodRecognition'

api = HfApi()

print("="*70)
print("📁 UPLOADING TO HUGGING FACE")
print("="*70)
print(f"Repository: https://huggingface.co/{REPO_ID}\n")

Files to upload
files = [

```

```

 ('best_model.pth', 'PyTorch model weights'),
 ('model_torchscript.pt', 'TorchScript model (mobile)'),
 ('model_info.json', 'Model metadata'),
 ('confusion_matrix.png', 'Confusion matrix'),
 ('per_class_accuracy.png', 'Per-class accuracy chart'),
 ('training_curves.png', 'Training curves'),
 ('classification_report.txt', 'Classification report'),
]

Upload README first
print("1. Uploading Model Card...")
try:
 api.upload_file(
 path_or_fileobj='/content/README.md',
 path_in_repo='README.md',
 repo_id=REPO_ID,
 repo_type='model',
)
 print(" 🟢 README.md uploaded")
except Exception as e:
 print(f" 🛑 Error: {e}")

Upload exported model folder
print("\n2. Uploading exported model files...")
export_dir = os.path.join(SAVE_DIR, 'exported_model')
if os.path.exists(export_dir):
 try:
 for file in os.listdir(export_dir):
 file_path = os.path.join(export_dir, file)
 if os.path.isfile(file_path):
 api.upload_file(
 path_or_fileobj=file_path,
 path_in_repo=file,
 repo_id=REPO_ID,
 repo_type='model',
)
 print(f" 🟢 {file}")
 except Exception as e:
 print(f" 🛑 Error: {e}")

Upload other files
print("\n3. Uploading results and visualizations...")
for filename, description in files:
 filepath = os.path.join(SAVE_DIR, filename)

 if not os.path.exists(filepath):
 print(f" ⚠️ Skipping {filename} (not found)")
 continue

 try:
 api.upload_file(
 path_or_fileobj=filepath,
 path_in_repo=filename,
 repo_id=REPO_ID,
 repo_type='model',
)
 size = os.path.getsize(filepath) / (1024 * 1024)
 print(f" 🟢 {filename:30s} ({size:.2f} MB)")
 except Exception as e:
 print(f" 🛑 {filename}: {e}")

print("\n" + "="*70)
print("🎉 UPLOAD COMPLETE!")
print("="*70)
print(f"\n🌐 Your model is now public at:")
print(f" https://huggingface.co/{REPO_ID}")
print(f"\n📄 View your model card:")
print(f" https://huggingface.co/{REPO_ID}/blob/main/README.md")
print(f"\n📖 Download instructions:")
print(f" from huggingface_hub import hf_hub_download")
print(f" model = hf_hub_download('{REPO_ID}', 'best_model.pth')")

```

=====

 UPLOADING TO HUGGING FACE

=====

Repository: <https://huggingface.co/ranasinghehashini/srilankan-food-recognition>

### 1. Uploading Model Card...

 README.md uploaded

### 2. Uploading exported model files...

Processing Files (1 / 1) : 100% 5.89MB / 5.89MB, 9.82MB/s

New Data Upload : 100% 5.89MB / 5.89MB, 9.82MB/s

...rted\_model/best\_model.pth: 100% 5.89MB / 5.89MB

 best\_model.pth

Processing Files (1 / 1) : 100% 2.00MB / 2.00MB, 17.2kB/s

New Data Upload : 100% 571kB / 571kB, 17.2kB/s

...odel/model\_torchscript.pt: 100% 2.00MB / 2.00MB

 model\_torchscript.pt

 model\_info.json

/usr/local/lib/python3.12/dist-packages/huggingface\_hub/hf\_api.py:9767: UserWarning: Warnings while validating metadata in repo card  
- empty or missing yaml metadata in repo card

warnings.warn(f"Warnings while validating metadata in README.md:\n{message}")

 README.md

### 3. Uploading results and visualizations...

Processing Files (1 / 1) : 100% 5.89MB / 5.89MB, 0.00B/s

New Data Upload : 0.00B / 0.00B, 0.00B/s

...ecognition/best\_model.pth: 100% 5.89MB / 5.89MB

...ecognition/best\_model.pth: 100% 5.89MB / 5.89MB

No files have been modified since last commit. Skipping to prevent empty commit.

WARNING:huggingface\_hub.hf\_api:No files have been modified since last commit. Skipping to prevent empty commit.

 best\_model.pth (5.62 MB)

 Skipping model\_torchscript.pt (not found)

 Skipping model\_info.json (not found)

Processing Files (1 / 1) : 100% 318kB / 318kB, 795kB/s

New Data Upload : 100% 318kB / 318kB, 795kB/s

...tion/confusion\_matrix.png: 100% 318kB / 318kB

...tion/confusion\_matrix.png: 100% 318kB / 318kB

 confusion\_matrix.png (0.30 MB)

Processing Files (1 / 1) : 100% 263kB / 263kB, 1.32MB/s

New Data Upload : 100% 263kB / 263kB, 1.32MB/s

...on/per\_class\_accuracy.png: 100% 263kB / 263kB

 per\_class\_accuracy.png (0.25 MB)

Processing Files (1 / 1) : 100% 387kB / 387kB, 1.94MB/s

New Data Upload : 100% 387kB / 387kB, 1.94MB/s

...tion/training\_curves.png: 100% 387kB / 387kB

 training\_curves.png (0.37 MB)

 classification\_report.txt (0.00 MB)

=====

 UPLOAD COMPLETE!

=====

 Your model is now public at:  
<https://huggingface.co/ranasinghehashini/srilankan-food-recognition>

 View your model card:  
<https://huggingface.co/ranasinghehashini/srilankan-food-recognition/blob/main/README.md>

 Download instructions:  
from huggingface\_hub import hf\_hub\_download  
model = hf\_hub\_download('ranasinghehashini/srilankan-food-recognition', 'best\_model.pth')

```
Cell 7: Test downloading from Hugging Face
import torch

from huggingface_hub import hf_hub_download

REPO_ID = "ranasinghehashini/srilankan-food-recognition"

print("Testing download from Hugging Face...")

Download model
```

```
Download model
model_path = hf_hub_download(
 repo_id=REPO_ID,
 filename="best_model.pth",
 cache_dir="/content/hf_cache"
)

print(f"✅ Model downloaded successfully!")
print(f" Location: {model_path}")

Verify it loads
checkpoint = torch.load(model_path, map_location='cpu')
print(f"\n✅ Model loads correctly!")
print(f" Epoch: {checkpoint['epoch']+1}")
print(f" Validation Accuracy: {checkpoint['val_acc']:.2f}%")
print(f" Classes: {checkpoint['classes']}")
```

Testing download from Hugging Face...

✅ Model downloaded successfully!  
Location: /content/hf\_cache/models--ranasinghehashini--srilankan-food-recognition/snapshots/16d17a20920ad38f1d

✅ Model loads correctly!  
Epoch: 52  
Validation Accuracy: 90.25%  
Classes: ['carrot\_raw', 'carrot\_white\_curry', 'greenbeans\_raw', 'greenbeans\_tempered', 'greenbeans\_white\_curry']